

Conceptualising Software Development Lifecycle for Engineering AI Planning Systems

Ilche Georgievski
Service Computing Department, IAAS
University of Stuttgart
Stuttgart, Germany

Abstract—Given the prominence of AI planning in research and industry, the development of AI planning software and its integration into production architectures are becoming important. However, building and managing planning software is a complex and expertise-dependent process without methodological support that would ensure AI planning applications have high quality and industrial strength. To that end, we propose a lifecycle for developing AI planning systems that consists of ten phases related to the design, development, and operation of planning systems.

Index Terms—AI Planning, AI Planning Software Development, AI Engineering, Software Engineering

I. INTRODUCTION

Artificial Intelligence (AI) planning is a long-standing research and development discipline concerned with solving planning problems that consist of an initial state of the world, a domain model in terms of actions that can change the world, and a goal state that represents the user goal. A solution to a planning problem is a course of actions that lead from the initial state to the goal. Nowadays, there is a growing demand for AI planning in real applications, such as space exploration [1], building automation [2], and autonomous driving [3]. These developments highlight the need for developing and integrating planning systems into existing software architectures.

In addition to the development of algorithms or domain models, engineering AI planning systems involves many complex activities: choosing the correct underlying planning model, knowledge engineering, dealing with numerous planning functionalities, architecting a system without an established interoperability mechanism for planning components, choosing suitable planning tools, and collecting and analysing relevant data [4]. As a result, building such planning systems is complex and demands immense knowledge about the application domain of interest, expertise in AI planning, and proficiency in software engineering. However, no methodology exists that covers and treats all relevant phases planning systems could and should go through. A common understanding of how planning system software is engineered is required; this will enable not only cooperation between involved parties but also facilitate the development of planning systems.

We introduce a general software development lifecycle model for AI planning systems that describes the relevant phases for engineering AI planning systems. The lifecycle can serve not only as a tool for planning application developers, but also as a basis for discussion in relevant fields.



Fig. 1. Software Development Lifecycle for AI Planning Systems.

II. AI PLANNING SOFTWARE DEVELOPMENT LIFECYCLE

Fig. 1 shows the *AI Planning Software Development Lifecycle* with ten phases planning systems should go through.

a) *Requirements Analysis*: This phase indicates that the development of AI planning systems must begin with the identification of requirements by the relevant stakeholders [5]. The requirements can be *functional requirements*, which include not only the required functionality but also the properties and restrictions that should be considered in the overall planning process; *non-functional requirements*, such as system performance, scalability, reusability, or maintainability; *goal-oriented requirements*, which define the objectives a planning system should achieve, including the knowledge about the planning domain [6]; and *user-related requirements*, which define conditions relevant for the user, e.g., planning systems should not put at risk the safety of humans.

b) *Planning Model Selection*: This phase is performed to decide what planning model is needed for the planning system. We define a *planning model* as a specific form of planning distinguished by a planning approach with specific assumptions, qualities, and conditions about the world. The decision should be based on all requirements from the previous phase and with special emphasis on the functional and goal-oriented requirements. This phase requires expertise and experience in planning, so the selection of a suitable planning model needs to be done by planning experts or supported by some tool.

c) *Domain Model Design*: The third phase is about creating a planning domain model. In general, the design process starts by deriving domain information from the requirements collected in the first phase. On the one hand, the domain information includes objects, properties, actions, and tasks, and, on the other hand, assumptions and features that are essential for the representation of the domain model. The domain information is then conceptualised and formalised in a suitable planning syntax (e.g., PDDL). The domain model can be fed in into a component with a solving functionality, assuming the component supports the chosen syntax [4].

d) *Architecture and Design*: This phase is concerned with conceptualising a system architecture that consists of suitable planning components and relationships between them based on the outcomes of previous phases. For example, a service-oriented architecture model could be a choice given the consideration of planning functionalities as planning services [7]. The planning model is essential here as it directs the conditions for an interaction model among planning components to ensure their compatibility. That is, the interaction model represents the *allowed* connections among the planning components considering various planning conditions, such as underlying data structures. The resulting architecture design should provide sufficient information for the selection and/or implementation of required components in the next phases.

e) *Planning Technology Selection*: Existing planning technology (e.g., code, tools, and services) should be explored and selected if available before entering the phase to implement required planning components. In this phase, existing planning technology is searched for to select appropriate implementations for the required planning functionalities. Places one could search in are limited though; some collections of planning technology can be found in [8], [9].

f) *Implementation*: The sixth phase is devoted to the implementation of the planning system following the architecture design. The implementation includes developing components for planning functionalities for which no available technology can be found and necessary communication mechanisms. For each planning component to be implemented, the classical software development lifecycle can be entered at this point.

g) *Testing*: In this phase, the planning system is tested to validate and verify its behaviour with respect to the requirements specified in the first phase. On the one hand, planning artefacts, such as planning components and domain models are tested in isolation. On the other hand, integration tests are needed to verify that all planning artefacts can work correctly together and satisfy the system-wide attributes.

h) *Deployment*: This phase is needed to make a planning system ready for execution. One can deploy planning components manually, which can be a time-consuming and error-prone process given the heterogeneity of planning technology [4]. Alternatively, the planning system deployment can be automated for which appropriate models are needed [4].

i) *Monitoring*: In this phase, the planning system and execution environment are monitored in order to understand the system and its constituent components, especially the cases

of problems and unexpected behaviour. Monitoring entails collecting provenance data specific to planning and storing the data for analysis, which is performed in the next phase.

j) *Analysis*: This last phase is dedicated to analysing the previously collected provenance data. It requires capabilities to identify problems, ask questions, and generate relevant insights to improve the planning system (e.g., domain knowledge), enable traceability, reproducibility, and explainability.

III. DISCUSSION

The AI planning software development lifecycle is a general process for engineering planning systems. As such, it might be necessary to further refine or adjust some of the phases to the specific needs of some application. For example, the *Domain Model Design* phase can be refined to include the steps of verification and validation of the domain model [10]. Some phases can drastically change due to new technological advancements. For example, the *Requirements Analysis* phase could include AI-driven requirements [11], or the *Planning Model Selection* phase may need to be updated when some method seems good at recommending relevant models.

Our AI planning software lifecycle provides the basis for discussions among practitioners and researchers of AI planning and AI engineering. It can be also used to identify open challenges in engineering planning systems. For example, one can observe a lack of support for selecting planning models, a registry of available planning tools, a toolbox of planning services, or a holistic strategy for testing planning systems.

REFERENCES

- [1] S. Chien and R. Morris, "Space Applications of Artificial Intelligence," *AI Magazine*, vol. 35, no. 4, pp. 3–6, 2014.
- [2] I. Georgievski, T. A. Nguyen, F. Nizamic, B. Setz, A. Lazovik, and M. Aiello, "Planning meets activity recognition: Service coordination for intelligent buildings," *Pervasive and Mobile Computing*, vol. 38, no. 1, pp. 110–139, 2017.
- [3] E. Alnazer, I. Georgievski, N. Prakash, and M. Aiello, "A Role for HTN Planning in Increasing Trust in Autonomous Driving," in *IEEE International Smart Cities Conference*, 2022, pp. 1–7.
- [4] I. Georgievski and U. Breitenbücher, "A Vision for Composing, Integrating, and Deploying AI Planning Functionalities," in *IEEE International Conference on Service-Oriented System Engineering*, 2021, pp. 166–171.
- [5] J. O. Grady, Ed., *System Requirements Analysis*, 2nd ed. Elsevier, 2014.
- [6] A. van Lamsweerde, *Requirements Engineering: From System Goals to UML Models to Software Specifications*. Wiley Publishing, 2009.
- [7] I. Georgievski, "Towards Engineering AI Planning Functionalities as Services," in *Service-Oriented Computing – ICSOC 2022 Workshops*, ser. Lecture Notes in Computer Science. Springer International Publishing, 2022.
- [8] I. Georgievski and M. Aiello, "HTN planning: Overview, comparison, and beyond," *Artificial Intelligence*, vol. 222, no. 0, pp. 124–156, 2015.
- [9] A. Green, B. J. Reji, C. M. ChrisE2018, E. Scala, F. Meneguzzi, F. M. Rico, H. Stairs, J. Dolejsi, M. Magnaguagno, and J. Mounty, "Planning.Wiki - The AI Planning & PDDL Wiki," 2019, accessed: January 24, 2023. [Online]. Available: <https://planning.wiki/>
- [10] J. R. Silva, J. M. Silva, and T. S. Vaquero, "Formal Knowledge Engineering for Planning: Pre and Post-Design Analysis," in *Knowledge Engineering Tools and Techniques for AI Planning*, M. Vallati and D. Kitchin, Eds. Springer International Publishing, 2020, pp. 47–65.
- [11] J. Bosch, H. Holmström Olsson, and I. Crnkovic, "It takes three to tango: Requirement, outcome/data, and AI driven development," in *International Workshop on Software-intensive Business: Start-ups, Ecosystems and Platforms*, S. Hyrynsalmi, A. Suominen, C. Jud, X. Wang, J. Bosch, and J. Münch, Eds., vol. 2305, 2018, pp. 177–192.